

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 03 – Konsep Dasar Pemrograman Berorientasi Objek



CUT GEUBRINA DWI MAHARANI

103132400034

S1SD-05-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (OOP) adalah paradigma pemrograman yang memaketkan data (atribut) dan fungsi (method) ke dalam satu kesatuan struktur terenkapsulasi yang saling berinteraksi, dengan empat karakteristik utama yaitu abstraksi, enkapsulasi, pewarisan, dan polimorfisme. Dalam penerapannya pada bahasa Java, struktur program dibangun dari cetak biru berupa kelas untuk menciptakan objek, di mana data disimpan melalui instance field (milik objek) atau class field (static, milik kelas), serta dikelola melalui method dan constructor dengan bantuan kata kunci this untuk merujuk pada objek yang sedang berjalan. Selain itu, modul ini menekankan perbedaan mekanisme pengiriman parameter, yaitu pass by value untuk tipe data primitif yang tidak mengubah data asal, serta pass by reference untuk tipe data objek dan array di mana perubahan data di dalam method akan langsung mempengaruhi objek aslinya di memori

B. Guided

1. [Abstraksi]

```
public class kucing {  
    String nama;  
    String ras;  
  
    void bersuara(){  
        System.out.println(nama + " " + ras + " " + "meow");  
    }  
}
```

```
public class abstraksi {  
  
    public static void main(String[] args) {  
  
        kucing miaw = new kucing();  
  
        miaw.nama = "Moci";  
        miaw.ras = "Scottish";  
    }  
}
```

```
miaw.bersuara();

}

}
```

Output :

```
Moci Scottish meow
```

2. [Enkapsulasi]

```
public class rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah){
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo (){
        System.out.println("Saldo Anda" + " " + saldo);
    }
}

public class enkapsulasi {

    public static void main(String[] args) {
        rekening BRI = new rekening();

        BRI.tambahSaldo(19750000);
        BRI.tampilkanSaldo();
    }
}
```

Output :

```
Saldo berhasil ditambahkan
Saldo Anda 19750000
```

3. [Field]

```
public class circle {
    public static final double PI = 3.14159;

    public static double radiansToDegrees (double rads) {
        return rads * 180 / PI;
    }
}
```

```

public double r;

public double area (){
    return PI * r * r;
}

public double circumference(){
    return 2 * PI * r;
}
}

```

4. [Keyword This]

```

public class Buku {
    private String pengarang;
    private String judul;
    private Buku() {
        this("Rumah Kita", "GoodBles");
        //this ini digunakan untuk memanggil konstruktor yang menerima dua parameter
    }
    private Buku(String judul,String pengarang)
    {
        this.judul = judul;
        this.pengarang = pengarang;
    }
    private void cetakKeLayar()
    {
        System.out.println("Judul : " + judul + " Pengarang : " + pengarang);
    }
    public static void main (String[] args)
    {
        Buku a,b;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}

```

Output :

```

Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles

```

```

public class lagu {
    private String pencipta;
    private String judul;
    public void isiparam(String judul, String pencipta) {
        this.judul = judul;
    }
}

```

```

        this.pencipta = pencipta;
    }
    public void cetakKeLayar() {
        if(judul == null && pencipta == null) return;
        System.out.println("Judul : " + judul + ", Pencipta : " + pencipta);
    }
}

public class DemoLagu {
    public static void main (String[] args) {
        lagu a = new lagu ();
        a.isiparam("God Will Make A Way", "Don Moen");
        a.cetakKeLayar();
    }
}

```

Output :

```
Judul : God Will Make A Way, Pencipta : Don Moen
```

5. [Method]

```

public class testpass {
    int i,j;

    testpass(int a, int b) {
        i = a;
        j = b;
    }
    //passed by value dengan parameter berupa tipe data primitf
    void calculate(int m, int n) {
        m = m*10;
        n = n/2;
    }
    //passed by reference dengan berupa tipe data class
    void calculate (testpass e){
        e.i = e.i*10;
        e.j = e.j/2;
    }
}

```

```

public class passbyvalueandreference {
    public static void main(String[] args) {
        int x,y;

        testpass z;
        z = new testpass(50,100);
        x = 10;
        y = 20;

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        //passed by value
        z.calculate(x,y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        //passed by reference
        z.calculate(z);

        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
    }
}

```

Output :

```

Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50

```

C. Unguided

1. Soal Unguided 1

Soal Unguided — Modul 3: Pemrograman Berorientasi Objek

Penerapan OOP Sederhana untuk Pengolahan Data

Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya.
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Program ini harus memuat dua jenis field: **instance field** (milik masing-masing object) dan **class field** (milik class, dipakai bersama).
- Tidak perlu inheritance, overriding, maupun abstract class.

Soal — Pembersihan dan Analisis Data Suhu Mingguan

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Kebutuhan program

Class ini harus dapat:

1. ****Menyimpan data suhu 7 hari****, dibuat dari array suhu mentah yang sudah ada di `main` (index array mewakili hari).
2. ****Menyimpan nilai penanda data kosong (`-1.0`)** sebagai aturan tetap milik class**, yaitu class field yang nilainya tidak dapat diubah dan dipakai bersama oleh semua object.
3. ****Menampilkan seluruh data apa adanya****, menandai hari yang kosong secara jelas.
4. ****Mencari apakah ada hari yang kosong, dan di index berapa****, karena index inilah yang nanti dipakai untuk proses pengisian data.
5. ****Mengisi hari yang kosong****, menggunakan index dari poin 4, dengan nilai perkiraan berdasarkan rata-rata suhu satu hari sebelum dan satu hari sesudahnya.
6. ****Menghitung rata-rata suhu**** dari data yang sudah bersih.

Hint

> Hint di bawah ini adalah acuan minimal. Anda tetap boleh menambah method atau menyesuaikan detail selama seluruh kebutuhan di atas terpenuhi.

****Nama class:**** `PengolahSuhu`

****Field yang disarankan:****

Field	Jenis	Tipe	Keterangan
---	---	---	---
`suhuHarian`	Instance field	`double[]`	Menyimpan array suhu 7 hari; berbeda untuk setiap object
`NILAI_KOSONG`	Class field	`double`	Nilai penanda data kosong (`-1.0`); sama untuk semua object dan tidak berubah

****Method yang disarankan:****

Method	Kebutuhan	Return
--------	-----------	--------

```

|---|---|---|
| `tampilkanData()` | 3 | `void` |
| `cariIndexKosong()` | 4 | `int` — index hari yang bernilai `NILAI_KOSONG`, atau
`-1` apabila tidak ditemukan |
| `isiDataKosong()` | 5 | `void` — memanggil `cariIndexKosong()` di dalamnya; jika
hasilnya bukan `-1`, isi index tersebut |
| `hitungRataRata()` | 6 | `double` |

```

****Rumus imputasi (pengisian data kosong) untuk time series:****

Untuk index hari `i` yang kosong:

```

...
suhuHarian[i] = ( suhuHarian[i-1] + suhuHarian[i+1] ) / 2
...

```

Ini adalah metode imputasi paling dasar untuk data deret waktu, yaitu ****mengisi nilai yang hilang dengan rata-rata dari titik data tetangga terdekatnya**** (sebelum dan sesudah), dengan asumsi nilai yang hilang berada di antara dua kondisi yang sudah diketahui dan tidak berubah drastis.

Data suhu (array di `main`)

Gunakan array berikut sebagai data suhu 7 hari (Hari 1 = index 0):

```

```java
double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };
...

```

Hari yang kosong: **\*\*Hari 4\*\***.

### Alur program di `main`

1. Siapkan array `suhuHarian` seperti di atas.
2. Buat object `PengolahSuhu` dari array `suhuHarian` tersebut. Tampilkan data awal (`tampilkanData()`), lalu panggil `cariIndexKosong()` dan tampilkan index yang ditemukan.
3. Jalankan `isiDataKosong()`.
4. Tampilkan data akhir yang sudah bersih (`tampilkanData()`) beserta rata-rata suhunya.

5. Tampilkan kembali isi array `suhuHarian` di `main` (bukan lewat object `PengolahSuhu`, melainkan variabel array yang sudah disiapkan di langkah 1).  
Amati bahwa array ini pun sudah berubah, padahal tidak pernah diubah langsung di `main`, lalu jelaskan sebabnya dalam beberapa kalimat sebagai komentar pada kode program.

### Output yang diharapkan

...

==== Data Suhu Awal ====

Hari 1 : 30.4°C

Hari 2 : 24.3°C

Hari 3 : 26.8°C

Hari 4 : (kosong)

Hari 5 : 31.4°C

Hari 6 : 30.8°C

Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

==== Data Suhu Setelah Pengisian ====

Hari 1 : 30.4°C

Hari 2 : 24.3°C

Hari 3 : 26.8°C

Hari 4 : 29.1°C

Hari 5 : 31.4°C

Hari 6 : 30.8°C

Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:

[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]

(ikut berubah: constructor menyimpan referensi array yang sama)

...

---

## Yang perlu diperhatikan saat mengerjakan

- Amati baik-baik dua baris terakhir pada output di atas. Array `suhuHarian` yang dicetak itu adalah variabel di `main`, bukan diambil lewat object `PengolahSuhu` — dan nilainya sudah berubah, padahal `main` tidak pernah memanggil operasi pengubahan itu sendiri. Ini terjadi karena array bertipe **\*\*reference\*\***: constructor `PengolahSuhu` menyimpan array yang sama persis (bukan menyalinnya), sehingga perubahan yang dilakukan `isiDataKosong()` pada object otomatis terlihat juga dari sisi `main`. Jelaskan hal ini dalam beberapa kalimat sebagai komentar pada kode program.
- Nilai penanda `-1.0` harus ditulis satu kali saja sebagai class field, lalu dipakai oleh setiap method yang membutuhkannya. Jangan mengetik ulang angka `-1.0` di dalam method.
- Rumus imputasi mengasumsikan hari sebelum dan sesudah hari yang kosong tersedia sebagai data asli (bukan ikut kosong). Data pada soal ini sudah dijamin memenuhi asumsi tersebut.

Jawaban :

```
public class PengolahSuhu {
 private double[] suhuHarian;

 public static final double NILAI_KOSONG = -1.0;

 public PengolahSuhu(double[] suhuHarian){
 this.suhuHarian = suhuHarian;
 }

 public void tampilkanData() {
 for (int i = 0; i < this.suhuHarian.length; i++){
 if(this.suhuHarian[i] == NILAI_KOSONG){
 System.out.println("Hari " + (i+1) + " : (kosong)");
 } else {
 System.out.println("Hari " + (i+1) + " : " + this.suhuHarian[i] + "°C");
 }
 }
 }
}
```

```

 }
}

public int cariIndexKosong(){
 for (int i = 0; i < this.suhuHarian.length; i++){
 if(this.suhuHarian[i] == NILAI_KOSONG){
 return i;
 }
 }
 return -1;
}

public void isiDataKosong(){
 int indexKosong = cariIndexKosong();

 if (indexKosong != 1 && indexKosong > 0 && indexKosong <
this.suhuHarian.length - 1){

 double suhuSebelum = this.suhuHarian[indexKosong - 1];
 double suhuSesudah = this.suhuHarian[indexKosong + 1];

 this.suhuHarian[indexKosong] = (suhuSebelum + suhuSesudah) / 2.0;
 }
}

public double hitungRataRata(){
 double total = 0;
 for(double suhu : this.suhuHarian) {
 total += suhu;
 }
 return total / this.suhuHarian.length;
}

```

```
}
```

```
import java.util.Arrays;
```

```
public class Main {
```

```
 public static void main(String[] args){
```

```
 double[] suhuHarian = {30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9};
```

```
 PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);
```

```
 System.out.println("=== Data Suhu Awal ===");
```

```
 pengolah.tampilkanData();
```

```
 System.out.println();
```

```
 int indexKosong = pengolah.cariIndexKosong();
```

```
 System.out.println("Index hari kosong (dimulai dari 0): " + indexKosong);
```

```
 System.out.println();
```

```
 pengolah.isiDataKosong();
```

```
 System.out.println("=== Data Suhu Setelah Pengisian ===");
```

```
 pengolah.tampilkanData();
```

```
 System.out.println();
```

```
 System.out.printf("Rata-rata : %.2f°C\n\n", pengolah.hitungRataRata());
```

```
 System.out.println("Isi array suhuHarian di main setelah isiDataKosong()
dijalankan:");
```

```

 System.out.println(Arrays.toString(suhuHarian));

 System.out.println("{ikut berubah: constructor menyimpan referensi array yang sama}");
 }
}

```

Output :

```

=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29,39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
{ikut berubah: constructor menyimpan referensi array yang sama)

```

Penjelasan :

1. `private double[] suhuHarian;`  
Menerapkan enkapsulasi menggunakan access modifier private agar variabel tidak bisa di akses dari luar class. Field bersifat instance field dimana setiap kita membuat objek baru pada class `PengolahSuhu`, maka masing-masing objek akan

memiliki data suhuHarian nya sendiri

2. `public static final double NILAI_KOSONG = -1.0;`  
Dibuatkan nilai kosong = -1.0 agar menghindari penggunaan angka secara berulang-ulang di berbagai method
3. `public PengolahSuhu(double[] suhuHarian) { this.suhuHarian = suhuHarian; }`  
Constructor berfungsi untuk menginisialisasi nilai awal objek PengolahSuhu diciptakan, parameter suhuHarian dibuat sama persis dengan field suhuHarian agar keyword this dapat digunakan untuk memperjelas bahwa kita merujuk pada field milik objek itu sendiri, bukan variabel parameter
4. `public void tampilkanData()`  
Melakukan perulangan pada setiap elemen array dan memeriksa apakah nilai pada hari tersebut sama dengan NILAI\_KOSONG. Jika ya, akan mencetak (kosong). Jika tidak, mencetak nilai suhu
5. `public int cariIndexKosong()`  
Mencari dan mengembalikan posisi index array di tempat NILAI\_KOSONG ditemukan. Jika ditemukan NILAI\_KOSONG, method ini akan langsung menghentikan perulangan dan mengembalikan posisi index-nya
6. `public void isiDataKosong()`  
Mengetahui letak data yang bernilai kosong dengan memanggil method cariIndexKosong lalu memastikan bahwa index tidak berada di posisi pertama ataupun terakhir. Melakukan metode imputasi dengan menghitung rata-rata hari sebelum indexKosong -1 dan hari sesudah indexKosong +1, lalu menimpa nilai NILAI\_KOSONG dengan hasil perhitungan tersebut
7. `public double hitungRataRata()`  
Menjumlahkan seluruh elemen didalam array suhuHarian menggunakan perulangan lalu mengembalikan hasil pembagian total/jumlah\_Hari sebagai rata-rata suhu
8. `double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 }; PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);`  
Array di main akan ikut berubah setelah pengolah.isiDataKosong() dijalankan karena array merupakan variabel yang menyimpan nilainya ke alamat memori (refrensi) sehingga ketika kita mengirim variabel suhuHarian ke constructor, objek PengolahSuhu menunjuk ke tempat penyimpanan yang sama persis. Maka ketika method isiDataKosong() mengubah isi elemen array, variabel suhuHarian di main juga akan berubah

#### D. Kesimpulan

Praktikum ini melatih saya memperjelas saya perihal konsep dasar OOP, di mana OOP mengelompokkan data dan method ke dalam satu kesatuan yang terenkapsulasi dengan tujuan agar keamanan data tidak dapat diakses atau diubah secara langsung dari luar

class. Penggunaan keyword `this` secara eksplisit diperuntukkan membedakan antara field milik objek dengan parameter method yang memiliki variabel sama. Selain itu Array di dalam bahasa Java merupakan tipe data referensi dimana pemanggilan method di dalam objek maka akan mengubah isi array di main secara otomatis